

CONSTRAINTS BASED- PROBLEMS

1. School Timetable Scheduling

A school needs to assign **4 teachers (T1, T2, T3, T4)** to **4 class periods (P1, P2, P3, P4)** such that:

Constraints:

- i. T1 cannot teach during Period 1.
- ii. T2 must teach before T4 ($P1 < P2 < P3 < P4$).
- iii. Only one teacher can be assigned to one period.
- iv. T3 cannot teach during Period 4.
- v. Every teacher must be assigned exactly one period.

Apply the **Backtracking Search Algorithm** to find a valid assignment with all intermediate steps and constraint checks.

State the final timetable.

SOLUTION:

Solution Using Backtracking Search

Backtracking is a search technique that assigns one teacher at a time. After each assignment, it checks whether all the constraints are satisfied. If a constraint is violated, it goes back (backtracks) and tries another assignment.

Step 1: Assign T1

Possible periods for T1 are P2, P3, and P4, since T1 cannot teach in P1.

Assign:

T1 → P2

Constraint Check

- T1 is not assigned to P1 ✓
- P2 is empty ✓

Current Assignment

Teacher Period

T1	P2
----	----

Step 2: Assign T2

Choose the earliest available period for T2.

Assign:

T2 → P1

Constraint Check

- P1 is available ✓
- T2 will still be before T4 ✓

Current Assignment

Teacher Period

T1 P2

T2 P1

Step 3: Assign T3

First, try assigning T3 → P4.

Constraint Check

- T3 cannot teach during P4 ✗

This assignment violates the constraint.

Backtracking

Reject T3 → P4 and try another period.

Now assign:

T3 → P3

Constraint Check

- T3 is not in P4 ✓
- P3 is available ✓

Current Assignment

Teacher Period

T1 P2

T2 P1

T3 P3

Step 4: Assign T4

The only remaining period is P4.

Assign:

T4 → P4

Constraint Check

- T2 is in P1 and T4 is in P4.
- Since P1 comes before P4, the condition T2 must teach before T4 is satisfied. ✓
- P4 is empty ✓

Current Assignment

Teacher Period

T1	P2
T2	P1
T3	P3
T4	P4

Final Constraint Verification

Constraint	Result
T1 cannot teach in P1	✓ Satisfied
T2 teaches before T4	✓ Satisfied
Only one teacher per period	✓ Satisfied
T3 cannot teach in P4	✓ Satisfied
Every teacher is assigned exactly one period	✓ Satisfied

Search Process

Start

— Assign T1 → P2 ✓
— Assign T2 → P1 ✓

|— Try T3 → P4 ✗ (Constraint violated)

| |

| |— Backtrack

|

|— Assign T3 → P3 ✓

|

|— Assign T4 → P4 ✓

Final Timetable

Period Teacher

P1 T2

P2 T1

P3 T3

P4 T4

CONCLUSION:

Using the Backtracking Search Algorithm, the valid timetable is:

- T2 → P1
- T1 → P2
- T3 → P3
- T4 → P4

All the given constraints are satisfied, and the solution is obtained after one backtracking step when the assignment T3 → P4 violated the constraint. This is a clear, step-by-step solution suitable for writing in a university examination.

2. Conference Presentation Scheduling

A conference organizer needs to assign **4 presenters (A, B, C, D)** to **4 presentation slots (Morning, Noon, Afternoon, Evening)** such that:

Constraints:

- Presenter A cannot present in the Morning session.
- Presenter B must present before Presenter D.

- iii. Only one presenter is allowed in each session.
- iv. Presenter C cannot present in the Evening session.
- v. Every presenter must be assigned exactly one session.

Apply the **Backtracking Search Algorithm** to determine a valid schedule.

Show all intermediate assignments, constraint checking, and backtracking steps.

State the final presentation schedule.

SOLUTION:

The Backtracking Search Algorithm assigns one presenter at a time. After each assignment, it checks whether all the constraints are satisfied. If a constraint is violated, it goes back (backtracks) and tries another available slot.

Step 1: Assign Presenter A

Since A cannot present in the Morning, choose the next available slot.

Assign:

A → Noon

Constraint Check

- A is not assigned to Morning. ✓
- Noon is available. ✓

Current Assignment

Presenter Session

A	Noon
---	------

Step 2: Assign Presenter B

Choose the earliest available session.

Assign:

B → Morning

Constraint Check

- Morning is available. ✓
- B must present before D, which is still possible. ✓

Current Assignment

Presenter Session

A Noon

B Morning

Step 3: Assign Presenter C

First, try:

C → Evening

Constraint Check

- C cannot present in the Evening. ✗

Constraint Violated

Backtrack and try another session.

Now assign:

C → Afternoon

Constraint Check

- C is not in Evening. ✓
- Afternoon is available. ✓

Current Assignment

Presenter Session

A Noon

B Morning

C Afternoon

Step 4: Assign Presenter D

The only remaining session is Evening.

Assign:

D → Evening

Constraint Check

- B is in Morning and D is in Evening.
- Morning comes before Evening, so B presents before D. ✓

- Evening is available. ✓

Current Assignment

Presenter Session

A	Noon
B	Morning
C	Afternoon
D	Evening

Constraint Verification

Constraint	Status
A cannot present in Morning	✓ Satisfied
B presents before D	✓ Satisfied
Only one presenter per session	✓ Satisfied
C cannot present in Evening	✓ Satisfied
Every presenter assigned exactly one session	✓ Satisfied

Backtracking Search Tree

Start

|

└─ Assign A → Noon ✓

|

└─ Assign B → Morning ✓

|

└─ Try C → Evening ✗ (Constraint violated)

| |

| └─ Backtrack

|

└─ Assign C → Afternoon ✓

|
└─ Assign D → Evening ✓

Final Presentation Schedule

Session	Presenter
Morning	B
Noon	A
Afternoon	C
Evening	D

CONCLUSION:

Using the Backtracking Search Algorithm, the valid presentation schedule is:

- Morning → B
- Noon → A
- Afternoon → C
- Evening → D

The algorithm initially tried assigning Presenter C to the Evening session, but this violated the given constraint. It then backtracked and assigned C to the Afternoon session. All constraints are satisfied, resulting in the final valid schedule.

3. Library Book Exhibition Scheduling

A library needs to assign **4 books (B1, B2, B3, B4)** to **4 exhibition shelves (S1, S2, S3, S4)** such that:

Constraints:

- i. **B1** cannot be placed on **Shelf S1**.
- ii. **B2** must be placed before **B4** ($S1 < S2 < S3 < S4$).
- iii. Only **one book** can be placed on each shelf.
- iv. **B3** cannot be placed on **Shelf S4**.
- v. Every book must be assigned exactly one shelf.

Apply the Backtracking Search Algorithm to find a valid arrangement. Show all intermediate assignments, constraint checking, and backtracking steps. State the final shelf arrangement.

SOLUTION:

The **Backtracking Search Algorithm** assigns one book to a shelf at a time. After each assignment, it checks whether all the constraints are satisfied. If any constraint is violated, the algorithm **backtracks** and tries another shelf.

Step 1: Assign B1

Since **B1 cannot be placed on Shelf S1**, choose the next available shelf.

Assign:

B1 → S2

Constraint Check

- B1 is not placed on S1. ✓
- S2 is available. ✓

Current Assignment

Book Shelf

B1 S2

Step 2: Assign B2

Choose the earliest available shelf.

Assign:

B2 → S1

Constraint Check

- S1 is available. ✓
- B2 can still be placed before B4. ✓

Current Assignment

Book Shelf

B1 S2

B2 S1

Step 3: Assign B3

First, try:

B3 → S4

Constraint Check

- B3 cannot be placed on S4. ✗

Constraint Violated

Backtrack

Reject **B3 → S4** and try another shelf.

Now assign:

B3 → S3

Constraint Check

- B3 is not placed on S4. ✓
- S3 is available. ✓

Current Assignment

Book Shelf

B1 S2

B2 S1

B3 S3

Step 4: Assign B4

The only remaining shelf is **S4**.

Assign:

B4 → S4

Constraint Check

- B2 is placed on S1 and B4 is placed on S4.
- Since **S1 comes before S4**, the condition **B2 must be placed before B4** is satisfied. ✓
- S4 is available. ✓

Current Assignment

Book Shelf

B1 S2

B2 S1

B3 S3

Book Shelf

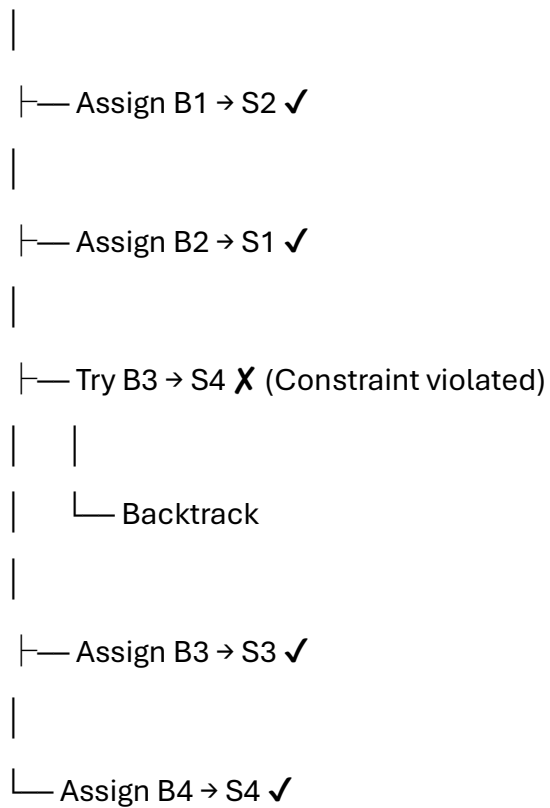
B4 S4

Constraint Verification

Constraint	Status
B1 cannot be placed on S1	✓ Satisfied
B2 is placed before B4	✓ Satisfied
Only one book on each shelf	✓ Satisfied
B3 cannot be placed on S4	✓ Satisfied
Every book is assigned exactly one shelf	✓ Satisfied

Backtracking Search Tree

Start



Final Shelf Arrangement

Shelf Book

S1 B2

Shelf Book

S2 B1

S3 B3

S4 B4

CONCLUSION:

Using the **Backtracking Search Algorithm**, the valid shelf arrangement is:

- **S1 → B2**
- **S2 → B1**
- **S3 → B3**
- **S4 → B4**

The algorithm first tried to place **B3** on **Shelf S4**, but this violated the given constraint. It then **backtracked** and placed **B3** on **Shelf S3**. All the constraints are satisfied, resulting in a valid exhibition shelf arrangement.

4. Flight Runway Scheduling

An airport controller needs to assign **4 flights (F1, F2, F3, F4)** to **4 landing slots (L1, L2, L3, L4)** such that:

Constraints:

- F1** cannot land in **Slot L1**.
- F2** must land before **F4**.
- Only **one flight** can use a landing slot.
- F3** cannot land in **Slot L4**.
- Every flight must be assigned exactly one landing slot.

Apply the Backtracking Search Algorithm to obtain a valid landing schedule. Show all intermediate assignments, constraint checking, and backtracking steps. State the final landing schedule.

SOLUTION:

The Backtracking Search Algorithm assigns one flight to a landing slot at a time. After each assignment, it checks whether all the constraints are satisfied. If any constraint is violated, it backtracks and tries another available slot.

Step 1: Assign F1

Since F1 cannot land in Slot L1, choose the next available slot.

Assign:

F1 → L2

Constraint Check

- F1 is not assigned to L1. ✓
- L2 is available. ✓

Current Assignment

Flight Landing Slot

F1 L2

Step 2: Assign F2

Choose the earliest available landing slot.

Assign:

F2 → L1

Constraint Check

- L1 is available. ✓
- F2 can still land before F4. ✓

Current Assignment

Flight Landing Slot

F1 L2

F2 L1

Step 3: Assign F3

First, try assigning:

F3 → L4

Constraint Check

- F3 cannot land in L4. ✗

Constraint Violated

Backtrack

Reject F3 → L4 and try another available slot.

Now assign:

F3 → L3

Constraint Check

- F3 is not assigned to L4. ✓
- L3 is available. ✓

Current Assignment

Flight Landing Slot

F1 L2

F2 L1

F3 L3

Step 4: Assign F4

The only remaining landing slot is L4.

Assign:

F4 → L4

Constraint Check

- F2 is in L1 and F4 is in L4.
- Since L1 comes before L4, the condition F2 must land before F4 is satisfied. ✓
- L4 is available. ✓

Current Assignment

Flight Landing Slot

F1 L2

F2 L1

F3 L3

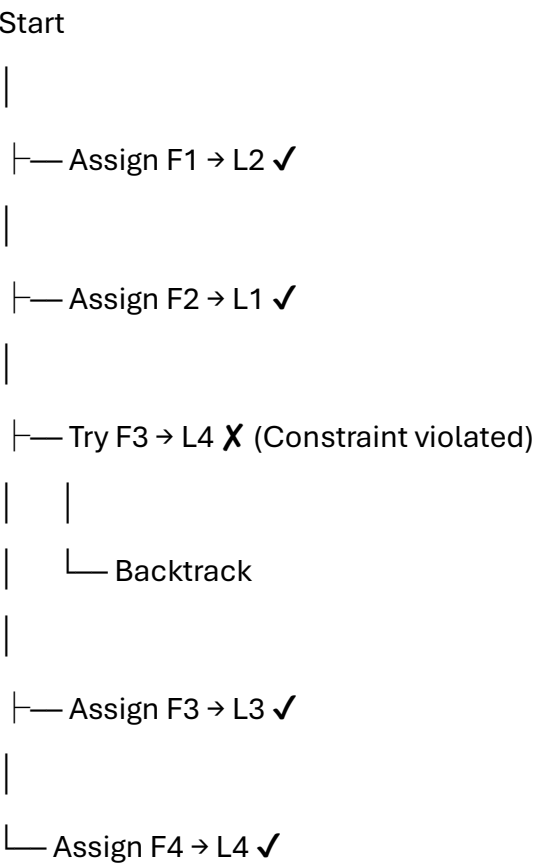
F4 L4

Constraint Verification

Constraint	Status
F1 cannot land in L1	✓ Satisfied

Constraint	Status
F2 lands before F4	✓ Satisfied
Only one flight per landing slot	✓ Satisfied
F3 cannot land in L4	✓ Satisfied
Every flight is assigned exactly one landing slot	✓ Satisfied

Backtracking Search Tree



Final Landing Schedule

Landing Slot	Flight
L1	F2
L2	F1
L3	F3
L4	F4

CONCLUSION:

Using the Backtracking Search Algorithm, the valid landing schedule is:

- L1 → F2
- L2 → F1
- L3 → F3
- L4 → F4

The algorithm first tried to assign F3 to Landing Slot L4, but this violated the given constraint. It then backtracked and assigned F3 to Landing Slot L3. All the constraints are satisfied, resulting in a valid landing schedule.

5. Cricket Match Batting Order

A cricket coach needs to assign **4 players (P1, P2, P3, P4)** to **4 batting positions (1, 2, 3, 4)** such that:

Constraints:

- P1** cannot bat at **Position 1**.
- P2** must bat before **P4**.
- Only **one player** can occupy each batting position.
- P3** cannot bat at **Position 4**.
- Every player must be assigned exactly one batting position.

Apply the Backtracking Search Algorithm to determine a valid batting order. Show all intermediate assignments, constraint checking, and backtracking steps. State the final batting order.

SOLUTION:

The Backtracking Search Algorithm assigns one player to a batting position at a time. After each assignment, it checks whether all the given constraints are satisfied. If any constraint is violated, it backtracks and tries another available position.

Step 1: Assign P1

Since P1 cannot bat at Position 1, choose the next available position.

Assign:

P1 → Position 2

Constraint Check

- P1 is not assigned to Position 1. ✓
- Position 2 is available. ✓

Current Assignment

Player Batting Position

P1 2

Step 2: Assign P2

Choose the earliest available position.

Assign:

P2 → Position 1

Constraint Check

- Position 1 is available. ✓
- P2 can still bat before P4. ✓

Current Assignment

Player Batting Position

P1 2

P2 1

Step 3: Assign P3

First, try assigning:

P3 → Position 4

Constraint Check

- P3 cannot bat at Position 4. ✗

Constraint Violated

Backtrack

Reject P3 → Position 4 and try another available position.

Now assign:

P3 → Position 3

Constraint Check

- P3 is not assigned to Position 4. ✓
- Position 3 is available. ✓

Current Assignment

Player Batting Position

P1 2

P2 1

P3 3

Step 4: Assign P4

The only remaining position is Position 4.

Assign:

P4 → Position 4

Constraint Check

- P2 is at Position 1 and P4 is at Position 4.
- Since Position 1 comes before Position 4, the condition P2 must bat before P4 is satisfied. ✓
- Position 4 is available. ✓

Current Assignment

Player Batting Position

P1 2

P2 1

P3 3

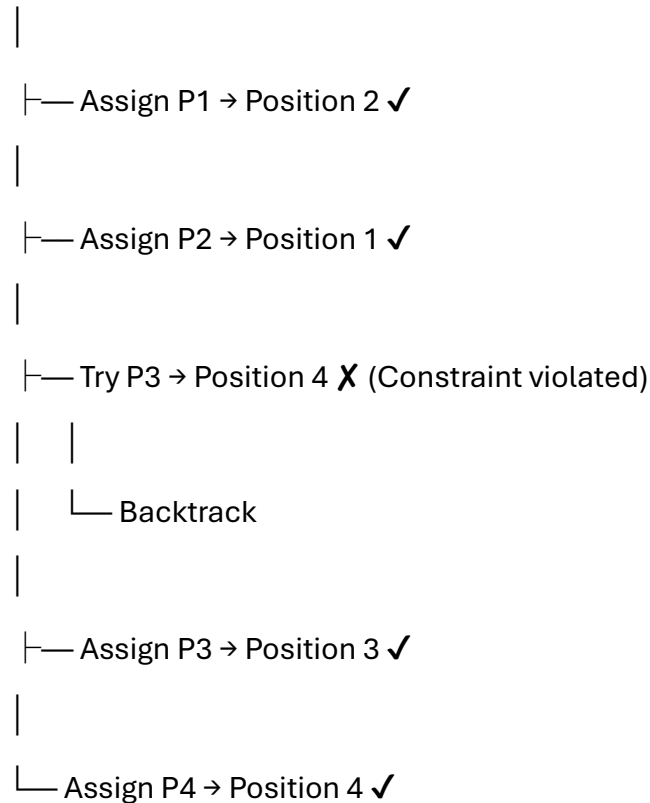
P4 4

Constraint Verification

Constraint	Status
P1 cannot bat at Position 1	✓ Satisfied
P2 bats before P4	✓ Satisfied
Only one player per batting position	✓ Satisfied
P3 cannot bat at Position 4	✓ Satisfied
Every player is assigned exactly one batting position	✓ Satisfied

Backtracking Search Tree

Start



Final Batting Order

Batting Position Player

1	P2
2	P1
3	P3
4	P4

CONCLUSION:

Using the Backtracking Search Algorithm, the valid batting order is:

- Position 1 → P2
- Position 2 → P1
- Position 3 → P3
- Position 4 → P4

The algorithm first tried assigning P3 to Position 4, but this violated the given constraint. It then backtracked and assigned P3 to Position 3. All the given constraints are satisfied, resulting in a valid batting order.

